

VIDEO DOCUMENTATION — OPENMANUS / AUTONOMOUS AI PLATFORM CONVERSATION

Source: Screen Recording 2026-09-23 105403.mp4

Length: approximately 5 minutes 42 seconds.

SCOPE NOTE

The recording begins in the middle of an existing ChatGPT response. This document captures the visible conversation hierarchy, headings, architecture diagrams, requirements, implementation details, and visible user/assistant turns. It is not presented as a word-for-word transcript of material that is outside the recording or not legible.

1. INITIAL ARCHITECTURAL IDEA — BUILDING A MANUS-LIKE AI

1.1 Core principle

- The agent architecture is the key.
- No need to train an LLM from scratch; an existing powerful model can be the “brain” with a custom agent layer around it.
- Proposed system: autonomous engineering + coding + research AI capable of reading documents, researching the internet, writing/executing code, analyzing results, creating diagrams, and producing finished projects.

1.2 Core architecture

YOUR AI → Reasoning + Memory → Planner → Browser / Python / Terminal → Verifier → Memory → Result.

A second diagram shows Browser / Python / Database → Verifier → Memory → Result.

1.3 Execution/build loop

- Build
- Run
- Test
- Inspect
- Diagnose
- Fix
- Repeat
- Deploy

The key idea is execution, observation, diagnosis, correction, and verification rather than code generation alone.

2. TARGET PRODUCT — MORE THAN A CHATBOT

Target: a general-purpose autonomous software engineer + browser agent + cloud computer + deployment platform.

Capabilities visible in the recording:

- Chat interface
- Autonomous planning
- Long-running tasks
- GitHub
- Git operations
- Browser
- Browser visualization
- Browser takeover
- Terminal
- Python execution
- Node.js
- Multiple languages
- File manipulation
- Code generation
- Code debugging
- Automatic testing
- Screenshot / visual testing
- Web research
- Database provisioning
- Docker
- Cloud VMs
- App deployment
- Custom domains
- SSL
- Persistent projects
- Background processes
- GitHub commits / PRs
- Project rollback
- Multi-agent orchestration
- Long-term memory
- Human takeover
- App monitoring

3. ROADMAP — EIGHT MAJOR PHASES

3.1 Phase 1 — Agent core

- Establish the autonomous loop first.
- PLAN → ACT → OBSERVE → THINK → ACT → VERIFY.
- Also shown as Plan → Act → Observe → Verify → Repeat.
- The agent understands a request, creates a plan, chooses tools, executes, observes results, updates the plan, detects failure, attempts correction, verifies, and reports completion.

3.2 Phase 2 — Give it a computer

Create an isolated cloud machine.

Example:

- Ubuntu VM
- Python
- Node.js
- Git
- Docker
- Chromium
- Java
- C/C++
- .NET
- Project files

Controlled access:

- terminal
- filesystem
- browser
- processes
- network
- Git

3.3 Phase 3 — Give it eyes

Browser actions:

- navigate()
- click()
- type()
- scroll()
- select()
- upload()
- download()
- screenshot()

Visual observation is required: the agent should receive screenshots and reason about visible application state.

3.4 Phase 4 — GitHub

Workflow:

- authorize
- select repository
- clone
- analyze codebase
- understand architecture
- create branch

- modify files
- run tests
- commit
- push
- pull request

3.5 Phase 5 — The coding agent

Required access:

- repository structure
- source files
- terminal
- compiler
- package manager

Workflow:

- implementation plan
- modify files
- run tests
- fix failures
- commit
- push / branch / PR

3.6 Phase 6 — Autonomous app creation

Example objective: “Create a SaaS platform for managing football academies.”

Breakdown:

- Requirements
- Architecture
- Database
- Backend
- Frontend
- Authentication
- Dashboard
- Payments
- Testing
- Deployment

3.7 Phase 7 — Hosting

Project → Project Manager → Provision VM / Container → Ubuntu / Docker / PostgreSQL / Redis / Node / Python / Nginx → Application.

Desired loop:

Build → deploy → test → monitor → fix → redeploy.

3.8 Phase 8 — Make it autonomous

Example:

“Build a complete trading dashboard and have it ready for users.”

Example task graph:

- Research requirements
- Design architecture
- Create repository
- Build frontend
- Build backend
- Create database
- Implement authentication
- Implement trading API

4. MULTI-AGENT / ORCHESTRATION

Specialized roles:

- Researcher → Research
- Coder → Engineer
- Browser → Tester
- Security
- Database
- DevOps

Orchestrator:

“What needs to happen?” → assign subtasks → agents execute → collect results → verify → correct → final result.

5. THE BIGGEST CHALLENGE

Reliable autonomous execution requires five things:

1. Intelligence — reasoning/coding models.

2. Tools — browser, GitHub, terminal, filesystem, APIs, databases, etc.

3. Environment — a real computer the AI can operate.

4. Feedback — see what happened after every action.

5. Persistence — memory, task state, files, repositories, running applications.

6. TARGET PRODUCT PRINCIPLE

The conversation rejects a lightweight Manus clone.

Target:

“A general-purpose autonomous software engineer + browser agent + cloud computer + deployment platform.”

Principles:

- no fallbacks / nothing less
- build the underlying autonomous computer first
- progressively add abilities
- avoid attempting hundreds of thousands of lines immediately
- make capabilities plug into one agent core

7. MODEL LAYER

YOUR AGENT → Model Adapter → Model A / Model B / Model C.

Purpose:

- avoid hard-wiring the platform to one provider
- upgrade intelligence without rebuilding the platform

8. DEVELOPMENT STARTING POINT

Do not start with the UI, hosting, or LLM training.

Start with:

- a tiny proof of concept
- give the AI a terminal
- verify reliable operation without manual intervention

Visible progression:

V1 — LLM + Agent loop + Terminal + Filesystem

V2 — Browser

V3 — GitHub

V4 — Autonomous debugging

V5 — broader autonomy

9. USER-SUPPLIED OPENMANUS REPOSITORIES

Visible user message:

“Hey there, I have the open source for the Manus AI app. Instead of starting from scratch, let's see how we can build it from there...”

Two ZIPs shown:

- OpenManus-main.zip
- OpenManus-RL-main.zip

10. TWO-REPOSITORY DISTINCTION

10.1 OpenManus

Agent runtime / application foundation:

- Agent
- Planning
- Tool calling
- Browser
- Python
- Shell
- Files
- MCP
- Sandbox
- Web search
- SWE agent

10.2 OpenManus-RL

Training / reinforcement-learning layer:

- Agent environments
- Rollouts
- Rewards
- PPO
- GRPO

- SFT
- Agent trajectories
- Benchmark environments
- RL training infrastructure

Proposed:

OUR PLATFORM

└─ OpenManus Core → Runtime agent

└─ OpenManus-RL → Training lab

↓

Improved models

RL is treated as a research/training layer, not the user-facing foundation.

11. MAJOR GAP — GITHUB

Missing/needed GitHub workspace capabilities:

- GitHub OAuth
- Repository browser
- Clone repository
- Branch management
- Commit
- Push
- Pull Request
- Issue management
- GitHub project workspace

GitHub becomes one of the first major systems to add.

12. EXISTING TOOL ARCHITECTURE

Existing:

User → LLM → ToolCallAgent → ToolCollection → Tools

Proposed:

OUR AGENT → ToolCallAgent → Tool Registry → GitHub / Browser / Terminal / Files / Cloud → Sandbox / VM → Build / Test → Deployment

13. SANDBOX / EXECUTION

Existing:

- SandboxManager

- DockerSandbox

Capabilities:

- create isolated containers
- execute commands
- read/write files
- manage working directories
- impose resource limits
- clean inactive sandboxes

Proposed:

User Project → Persistent Cloud Workspace → Git / Terminal / Browser → Docker / VM → Application

14. BROWSER SYSTEM

The existing repository is described as already integrating Browser Use CLI 3.0 through MCP.

Product interface:

Browser

└─ AI control

└─ Human control

↓

Browser View

Later:

Human UI ↔ Shared Browser Session ↔ OpenManus Agent

Agent capabilities:

- navigate URLs
- click elements
- type
- press keyboard keys
- extract page content
- wait
- take screenshots
- receive screenshots as visual tool output

Protections:

- browser session IDs attached to persistent tasks
- automatic browser attachment when starting a task

15. SOFTWARE-ENGINEERING AGENT

OpenManus contains:

app/agent/swe.py

Underlying capabilities:

- shell tools
- file editing
- tool calling
- execution
- LLM reasoning

Proposed:

Compiler → Tests → Debugger → Fix code → Retest.

The agent should be outcome-driven, not merely code-generating.

16. WHAT NOT TO DO

Rejected:

OpenManus → add 100 random features → huge tangled codebase → hope it becomes Manus.

Instead: refactor OpenManus into a proper platform.

17. PROPOSED PLATFORM REPOSITORY STRUCTURE

tools/

└─ github/

└─ browser/

└─ terminal/

└─ filesystem/

└─ search/

└─ database/

└─ deployment/

computer/

└─ browser/

└─ desktop/

└─ terminal/

└ screen/
sandbox/
└─ docker/
└─ vm/
└ security/
projects/
└─ workspace/
└─ builds/
└ deployments/
cloud/
└─ provisioning/
└─ deployment/
└─ domains/
└ monitoring/
memory/
└─ short_term/
└─ long_term/
└ project_memory/

18. OPENMANUS-RL IMPROVEMENT LOOP

Task → Agent actions → Tool calls → Observations → Errors → Corrections → Successful result.

Example:

1000 coding tasks → Agent attempts → Successful trajectories → Failures + recovery trajectories → Reward signals.

Purpose: continuously improve agent performance.

19. CHANGE TO THE ORIGINAL PLAN

Earlier:

- Start with V1 from scratch.

Updated:

- OpenManus = foundation
- OpenManus-RL = training/research layer
- Our project = product layer + major extensions

20. FIRST MILESTONE

“Perform a complete architectural audit of the two repositories.”

Audit areas:

- agent architecture
- planning / flow
- tools
- MCP
- browser integration
- sandbox
- SWE agent
- configuration
- entry points
- protocols
- RL environments
- memory
- algorithms
- project structure

21. AUDIT RESULT

OpenManus is summarized as:

Agents → Manus / Browser / SWE / MCP / Data / Sandbox

Tools → Python / Shell / Files / Search / Browser / Visualization

Flows → Planning / Multi-agent

↓

Sandbox → Docker / Daytona

Normal experience:

python main.py → create Manus → agent.run(prompt)

The visible assessment says there is an A2A server/protocol implementation, but not a complete Manus-style consumer application with the persistent project/workspace/cloud UI being proposed.

Transformation:

OpenManus agent framework → Autonomous AI Computer + Software Engineering + Cloud Development Platform

22. AUDIT STATUS

Visible conclusions:

- Core agent — strong foundation → extend
- Agent evaluation — RL repository → expand
- Agent training — RL foundation → integrate later
- Self-improvement — research foundation → later phase

23. AGENT CORE TO PRESERVE

BaseAgent → ToolCallAgent → Manus

Tool abstraction:

ToolCollection → BaseTool → individual tools

24. REQUIRED SWE WORKFLOW

Software Engineer

└─ Repository → Code analysis

└─ Terminal → Build

└─ Browser → UI testing

↓

Tests

└─ PASS

└─ FAIL → Diagnose → Fix code → Rebuild

↓

Verify

25. PRODUCTION VS TRAINING

RL infrastructure includes:

- memory
- rollouts
- algorithms
- SFT
- GRPO/PPO-related infrastructure
- reward concepts
- benchmark datasets
- Verl integration

- environment managers

Production should not simply be:

User → GRPO → Agent

Instead:

Production Platform → Agent → Task trajectory → Evaluation system

26. VERSION 0.2 — VISIBLE IMPLEMENTATION

Visible heading:

“15. Version 0.2”

Visible request:

“Take my GitHub repository, add this feature, test it, and create a pull request.”

Workflow:

OAuth → Repository → Clone → Analyze → Branch → Implement → Test → Browser test → Fix → Commit → Push → PR

Visible implementation details:

- clone repository into the project’s isolated workspace
- private-repository authentication through GITHUB_TOKEN
- create working branches
- view git status
- view code diffs
- commit changes
- push branches back to GitHub
- project tracks repository, current branch, and Git readiness
- Git credentials remain server-side
- automated tests were added; the visible message says three platform tests passed

Transition:

“AI agent that works in a local workspace”

→

“AI developer that can take ownership of an actual GitHub project.”

27. DURABLE PROJECT / TASK SYSTEM

Database → persistent projects → persistent tasks → checkpoints → resume after interruption → task history.

28. SHARED BROWSER / PLATFORM UI

Earlier:

Human → Platform Browser

Agent → separate Browser Use / MCP browser

Updated:

Human UI ↔ Shared Browser Session ↔ OpenManus Agent

Visible local UI:

`http://127.0.0.1:8000`

29. CONFIGURING THE AI MODEL

Visible heading:

“2. Configure the AI model”

Examples:

`OPENAI_API_KEY=your_key`

`GITHUB_TOKEN=your_github_token`

The keys should remain server-side and not be put in the frontend.

30. FINAL SYSTEM BEHAVIOR

Example:

“Build me a modern e-commerce platform for selling sneakers. It needs authentication, product management, cart, checkout, admin dashboard, PostgreSQL database, responsive frontend, and payment integration. Deploy it when all tests pass.”

System:

- Understand requirements
- Create project
- Plan architecture
- Create files
- Install dependencies
- Implement frontend
- Implement backend
- Run tests
- Deploy after successful verification

31. FINAL VISION

A persistent autonomous development platform where a user can provide a high-level objective and the system can:

- understand it
- plan it
- create a project
- operate a computer/browser
- work with GitHub
- write and modify code
- execute builds
- test applications
- inspect visual output
- diagnose failures
- correct them
- maintain persistent project state
- provision cloud infrastructure
- deploy
- monitor
- redeploy after fixes
- use evaluation/training infrastructure to improve the agent

32. LAST VISIBLE EXCHANGE

User:

“Produce a clean DOCX containing the entire transcript”

Assistant response visible in the recording:

It explains that a complete verbatim transcript could not safely be produced from the then-available chat context and requests the full exported conversation for an exact transcript.

The present document instead records the conversation content visible in the supplied video.